



# Chapter 3 Functions

CS 101- Programming  
Fundamentals

Instructor: Neelma Bhatti

September 8 – September 20, 2022

# Function

Named sequence of statements that performs a computation.

- Function *call* : *Takes* an argument and *returns* a result.
- **Void functions :**
  - Might display something on the screen or have some other effect
  - Do not have a return value.

# Function

Named sequence of statements that performs a computation.

1) `print('Number is odd')`

2) `input('Enter your marks')`

3) `type(3.45)`

- ***Name:*** print, input, type
- ***Arguments:*** Number is odd, Enter your marks, Hello world
- ***Return value:*** void, string, class type of argument.

# Why functions?

- Eliminating repetitive code.
- Naming a block of code for easier readability and debugging.
- Debug the parts one at a time and then assemble them into a working whole.
- Reusability in other programs and modules.

# Math functions

- **Module:** file that contains a collection of related functions.
- **Import statement** creates a module object.

```
>>> import math
```

```
>>> math
```

```
<module 'math' (built-in)>
```

# Accessing functions

- **Dot notation:** name of the module (dot/period) name of the function.

```
>>> math.sqrt
```

```
>>> math.log10
```

```
>>> math.sin
```

- **Dir(ModuleName):** Lists down all the functions present in a Python module.

# Adding/Creating new functions

**Function definition:** specifies the *name* of a new function and the *sequence of statements* that run when the function is called.

- Same naming rules and conventions as variables
- **def** keyword indicates a function definition.

```
def welcome():  
    print ( "Welcome to Habib " + first_name + last_name)
```

# Function

## Sequence of instructions for getting lunch from Tapal café:

- Get cash/card.
- Choose a menu item.
- If you ordered biryani, pay 250. If you ordered Daal, pay 180.
- Receive change from counter.

# Function

Sequence of instructions for getting lunch from Tapal café:

```
cash = 500 #Get cash
menu_item = input ("What do you want to order?) #Choose a menu item.
#If you ordered biryani, pay 250. If you ordered Daal, pay 180.
if menu_item == 'biryani':
    change = cash - 250
elif menu_item == 'daal':
    change = cash - 180
else:
    print ('Menu item unavailable')
#Receive change from counter.
print (change)
```

# Function

**Named** sequence of instructions for getting lunch from Tapal café:

```
Def get_lunch_tapal ( )
```

```
    cash = 500 #Get cash
```

```
    menu_item = input ("What do you want to order?) #Choose a menu item.
```

```
#If you ordered biryani, pay 250. If you ordered Daal, pay 180.
```

```
    if menu_item == 'biryani':
```

```
        change = cash - 250
```

```
    elif menu_item == 'daal':
```

```
        change = cash - 180
```

```
#Receive change from counter.
```

```
    print ('Enjoy your ' + menu_item)
```

```
    print (change)
```

# Flow of execution

- Flow of execution:
  - Begins at the first statement of the program.
  - Statements executed one at a time, from top to bottom.
- Arguments vs parameters
- Local vs global scope

# Composition

Ability to take small building blocks (*variables, expressions, statements*) and **compose** them.

- Arguments can be:
  - Values or variables.
  - Any kind of expression, including arithmetic operators.
  - Calls to other functions.